

150 DSA Problems, 15 Patterns

A Complete Guide with Explanations and Python Implementations

Introduction

The 15 Patterns

Pattern 1: Two Pointers

- 1.1 Pair with Target Sum
- 1.2 Remove Duplicates from Sorted Array
- 1.3 Squaring a Sorted Array
- 1.4 Triplet Sum to Zero (3Sum)
- 1.5 Triplet Sum Close to Target
- 1.6 Triplets with Smaller Sum
- 1.7 Subarrays with Product Less Than Target
- 1.8 Dutch National Flag Problem (Sort Colors)
- 1.9 Trapping Rain Water
- 1.10 Container With Most Water

Pattern 2: Sliding Window

- 2.1 Maximum Sum Subarray of Size K
- 2.2 Smallest Subarray with a Given Sum
- 2.3 Longest Substring with K Distinct Characters
- 2.4 Fruits into Baskets
- 2.5 Longest Substring Without Repeating Characters
- 2.6 Longest Substring with Same Letters after Replacement
- 2.7 Longest Subarray with Ones after Replacement
- 2.8 Permutation in a String
- 2.9 String Anagrams (Find All Anagram Start Indices)
- 2.10 Minimum Window Substring

Pattern 3: Fast & Slow Pointers

- 3.1 LinkedList Cycle Detection
- 3.2 Start of LinkedList Cycle
- 3.3 Happy Number
- 3.4 Middle of the LinkedList
- 3.5 Palindrome LinkedList
- 3.6 Rearrange a LinkedList (Odd-Even Interleave Reversal)
- 3.7 Cycle in a Circular Array
- 3.8 Find the Duplicate Number
- 3.9 Remove Nth Node From End of List
- 3.10 Reorder List (LeetCode variant using fast/slow explicitly)

Pattern 4: Merge Intervals

- 4.1 Merge Intervals
- 4.2 Insert Interval
- 4.3 Intervals Intersection
- 4.4 Conflicting Appointments (Can Attend All Meetings)
- 4.5 Minimum Meeting Rooms
- 4.6 Maximum CPU Load
- 4.7 Employee Free Time
- 4.8 Non-overlapping Intervals
- 4.9 Interval List Intersections (Merge and Count Overlaps)

4.10 Partition Labels

Pattern 5: Cyclic Sort

- 5.1 Cyclic Sort
- 5.2 Find the Missing Number
- 5.3 Find All Missing Numbers
- 5.4 Find the Duplicate Number (1..n-1 range)
- 5.5 Find All Duplicate Numbers
- 5.6 Find the Corrupt Pair (One Missing, One Duplicate)
- 5.7 First Missing Positive
- 5.8 Kth Missing Positive Number
- 5.9 Set Mismatch
- 5.10 Smallest Missing Positive After Replacing Negatives

Pattern 6: In-place Reversal of a LinkedList

- 6.1 Reverse a LinkedList
- 6.2 Reverse a Sub-list (Between Positions p and q)
- 6.3 Reverse Every K-element Sub-list
- 6.4 Reverse Alternating K-element Sub-list
- 6.5 Rotate a LinkedList
- 6.6 Swap Nodes in Pairs
- 6.7 Reverse Nodes in Even Length Groups
- 6.8 Palindrome Check via In-place Reversal
- 6.9 Add Two Numbers as LinkedLists
- 6.10 Split LinkedList in Parts

Pattern 7: Tree Breadth-First Search (BFS)

- 7.1 Binary Tree Level Order Traversal
- 7.2 Reverse Level Order Traversal
- 7.3 Zigzag Traversal
- 7.4 Level Averages in Binary Tree
- 7.5 Minimum Depth of Binary Tree
- 7.6 Maximum Depth of Binary Tree (BFS)
- 7.7 Level Order Successor
- 7.8 Connect Level Order Siblings
- 7.9 Right View of Binary Tree
- 7.10 Binary Tree Level Order Traversal II (Bottom-up)

Pattern 8: Tree Depth-First Search (DFS)

- 8.1 Binary Tree Path Sum
- 8.2 All Paths for a Sum
- 8.3 Sum of Path Numbers
- 8.4 Path With Given Sequence
- 8.5 Count Paths for a Sum
- 8.6 Diameter of Binary Tree
- 8.7 Lowest Common Ancestor
- 8.8 Path Sum III (Count Subpaths, Not Necessarily Root-Rooted)
- 8.9 Binary Tree Maximum Path Sum
- 8.10 Serialize and Deserialize Binary Tree

Pattern 9: Two Heaps

- 9.1 Find the Median of a Number Stream
- 9.2 Sliding Window Median
- 9.3 Maximize Capital (IPO Problem)
- 9.4 Next Interval
- 9.5 Task Scheduler
- 9.6 Meeting Rooms II (Heap-based)
- 9.7 Kth Largest Element in a Stream
- 9.8 Rearrange String K Distance Apart

9.9 Maximum Frequency Stack

9.10 Trapping Rain Water II

Pattern 10: Subsets / Backtracking

10.1 Subsets

10.2 Subsets II (With Duplicates)

10.3 Permutations

10.4 Permutations II (With Duplicates)

10.5 Combination Sum

10.6 Combination Sum II (No Reuse, With Duplicates)

10.7 Letter Case Permutation

10.8 Generate Parentheses

10.9 Word Search

10.10 N-Queens

Pattern 11: Modified Binary Search

11.1 Order-agnostic Binary Search

11.2 Ceiling of a Number

11.3 Next Letter

11.4 Number Range (First and Last Position)

11.5 Search in a Sorted Infinite Array

11.6 Minimum Difference Element

11.7 Search Bitonic Array

11.8 Search in Rotated Sorted Array

11.9 Find K Closest Elements

11.10 Median of Two Sorted Arrays

Pattern 12: Top K Elements (Heap)

12.1 Top K Numbers

12.2 Kth Smallest Number

12.3 K Closest Points to Origin

12.4 Connect Ropes with Minimum Cost

12.5 Top K Frequent Numbers

12.6 Frequency Sort

12.7 Kth Largest Number in a Stream

12.8 K Closest Numbers (to a Given Value)

12.9 Maximum Distinct Elements

12.10 Sum of Elements Between K1 and K2 Smallest

Pattern 13: K-way Merge

13.1 Merge K Sorted Lists

13.2 Kth Smallest Number in M Sorted Lists

13.3 Kth Smallest Number in a Sorted Matrix

13.4 Smallest Number Range Covering K Lists

13.5 K Pairs with Largest Sums

13.6 Merge K Sorted Arrays

13.7 Find K Pairs with Smallest Sums

13.8 Kth Smallest Element in a BST

13.9 Median of K Sorted Arrays

13.10 Find Common Elements in K Sorted Arrays

Pattern 14: Dynamic Programming (Knapsack Family)

14.1 0/1 Knapsack

14.2 Equal Subset Sum Partition

14.3 Subset Sum

14.4 Minimum Subset Sum Difference

14.5 Count of Subset Sum

14.6 Target Sum

14.7 Unbounded Knapsack (Rod Cutting)

- 14.8 Coin Change (Minimum Coins)
- 14.9 Coin Change II (Count Combinations)
- 14.10 Longest Common Subsequence

Pattern 15: Graphs (BFS / DFS / Union-Find / Topological Sort)

- 15.1 Number of Islands
- 15.2 Clone Graph
- 15.3 Course Schedule (Cycle Detection via Topological Sort)
- 15.4 Course Schedule II (Return the Order)
- 15.5 Number of Connected Components (Union-Find)
- 15.6 Graph Valid Tree
- 15.7 Redundant Connection
- 15.8 Word Ladder (BFS Shortest Transformation)
- 15.9 Rotting Oranges (Multi-source BFS)
- 15.10 Dijkstra's Shortest Path

Closing Notes

Introduction

This guide organizes **150 hand-picked Data Structures & Algorithms problems** into **15 core patterns**. Once you recognize a pattern, you can solve dozens of superficially different interview problems with the same underlying technique.

For each pattern you get:

- A short explanation of **when and why** the pattern applies
- **10 representative problems**, each with:
 - Problem statement
 - Key approach / intuition
 - A clean **Python** implementation
 - Time and space complexity

How to use this guide: Don't just read the code. For every problem, try to solve it yourself first (even just on paper) before looking at the approach. Pattern recognition is a skill built through repetition, not memorization.

| The 15 Patterns

1. Two Pointers
2. Sliding Window
3. Fast & Slow Pointers
4. Merge Intervals
5. Cyclic Sort
6. In-place Reversal of a LinkedList
7. Tree Breadth-First Search (BFS)
8. Tree Depth-First Search (DFS)
9. Two Heaps
10. Subsets / Backtracking
11. Modified Binary Search
12. Top K Elements (Heap)
13. K-way Merge
14. Dynamic Programming (Knapsack family)
15. Graphs (BFS / DFS / Union-Find / Topological Sort)

Pattern 1: Two Pointers

When to use it: You have a sorted array/list (or can sort it) and need to find pairs, triplets, or ranges that satisfy a sum/order condition. Instead of nested loops ($O(n^2)$ or worse), use two indices moving toward or away from each other to cut the search space linearly.

1.1 Pair with Target Sum

Problem: Given a sorted array, find a pair whose sum equals a target.

Approach: Start `left` at 0 and `right` at the end. If the sum is too small, move `left` right; if too big, move `right` left.

```
def pair_with_target_sum(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        current = arr[left] + arr[right]
        if current == target:
            return [left, right]
        if current < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]
```

Complexity: $O(n)$ time, $O(1)$ space.

1.2 Remove Duplicates from Sorted Array

Problem: Remove duplicates in-place from a sorted array and return the new length.

Approach: `next_non_dup` marks where the next unique element should go; a second pointer scans forward.

```
def remove_duplicates(arr):
    next_non_dup = 1
    for i in range(1, len(arr)):
        if arr[next_non_dup - 1] != arr[i]:
            arr[next_non_dup] = arr[i]
            next_non_dup += 1
    return next_non_dup
```

Complexity: $O(n)$ time, $O(1)$ space.

1.3 Squaring a Sorted Array

Problem: Given a sorted array of positive and negative numbers, return the squares sorted in non-decreasing order.

Approach: The largest squares come from the extremes. Fill the result array from the back using two pointers.

```
def make_squares(arr):
    n = len(arr)
    squares = [0] * n
    left, right = 0, n - 1
    highest_index = n - 1
    while left <= right:
        left_sq, right_sq = arr[left] ** 2, arr[right] ** 2
        if left_sq > right_sq:
            squares[highest_index] = left_sq
            left += 1
        else:
            squares[highest_index] = right_sq
            right -= 1
            highest_index -= 1
    return squares
```

Complexity: $O(n)$ time, $O(n)$ space.

1.4 Triplet Sum to Zero (3Sum)

Problem: Find all unique triplets in an array that sum to zero.

Approach: Sort the array. Fix one element, then use two pointers on the rest to find pairs summing to its negation, skipping duplicates.

```
def search_triplets(arr):
    arr.sort()
    triplets = []
    for i in range(len(arr) - 2):
        if i > 0 and arr[i] == arr[i - 1]:
            continue
        left, right = i + 1, len(arr) - 1
        while left < right:
            total = arr[i] + arr[left] + arr[right]
            if total == 0:
                triplets.append([arr[i], arr[left], arr[right]])
                left += 1
                right -= 1
                while left < right and arr[left] == arr[left - 1]:
                    left += 1
                while left < right and arr[right] == arr[right + 1]:
                    right -= 1
            elif total < 0:
                left += 1
            else:
                right -= 1
    return triplets
```

Complexity: $O(n^2)$ time, $O(n)$ space (sort).

1.5 Triplet Sum Close to Target

Problem: Find a triplet whose sum is closest to a given target.

Approach: Same skeleton as 3Sum, but track the smallest absolute difference seen so far.

```
def triplet_sum_close_to_target(arr, target_sum):
    arr.sort()
    smallest_diff = float('inf')
    result = 0
    for i in range(len(arr) - 2):
        left, right = i + 1, len(arr) - 1
        while left < right:
            target_diff = target_sum - arr[i] - arr[left] - arr[right]
            if target_diff == 0:
                return target_sum
            if abs(target_diff) < abs(smallest_diff):
                smallest_diff = target_diff
                result = arr[i] + arr[left] + arr[right]
            if target_diff > 0:
                left += 1
            else:
                right -= 1
    return result
```

Complexity: $O(n^2)$ time, $O(n)$ space.

1.6 Triplets with Smaller Sum

Problem: Count triplets whose sum is less than a target.

Approach: Fix one element; for the remaining two-pointer window, every step where the sum is small contributes `right - left` valid triplets at once.

```
def triplet_with_smaller_sum(arr, target):
    arr.sort()
    count = 0
    for i in range(len(arr) - 2):
        left, right = i + 1, len(arr) - 1
        while left < right:
            if arr[i] + arr[left] + arr[right] < target:
                count += right - left
                left += 1
            else:
                right -= 1
    return count
```

Complexity: $O(n^2)$ time, $O(n)$ space.

1.7 Subarrays with Product Less Than Target

Problem: Return all contiguous subarrays whose product is less than a target.

Approach: Maintain a sliding window with a running product; shrink from the left whenever the product is too big, then emit every suffix of the current window.

```
from collections import deque

def find_subarrays(arr, target):
    result = []
    left, product = 0, 1.0
    window = deque()
    for right in range(len(arr)):
        window.append(arr[right])
        product *= arr[right]
        while product >= target and left < right:
            product /= window.popleft()
            left += 1
        if product < target:
            temp = list(window)
            for i in range(len(temp) - 1, -1, -1):
                result.append(list(temp[i:]))
    return result
```

Complexity: $O(n^3)$ worst case for building output, $O(n^2)$ for the window logic; $O(n)$ extra space excluding output.

1.8 Dutch National Flag Problem (Sort Colors)

Problem: Sort an array containing only 0s, 1s, and 2s in-place.

Approach: Use three pointers `low`, `i`, `high`; swap 0s toward the front and 2s toward the back in a single pass.

```
def dutch_flag_sort(arr):
    low, i, high = 0, 0, len(arr) - 1
    while i <= high:
        if arr[i] == 0:
            arr[i], arr[low] = arr[low], arr[i]
            i += 1
            low += 1
        elif arr[i] == 1:
            i += 1
        else:
            arr[i], arr[high] = arr[high], arr[i]
            high -= 1
    return arr
```

Complexity: O(n) time, O(1) space.

1.9 Trapping Rain Water

Problem: Given elevation heights, compute how much water can be trapped after rain.

Approach: Track the maximum height seen so far from the left and right; move the pointer on the side with the smaller max, adding trapped water as you go.

```
def trapping_rain_water(heights):
    if not heights:
        return 0
    left, right = 0, len(heights) - 1
    left_max, right_max = heights[left], heights[right]
    water = 0
    while left < right:
        if left_max < right_max:
            left += 1
            left_max = max(left_max, heights[left])
            water += left_max - heights[left]
        else:
            right -= 1
            right_max = max(right_max, heights[right])
            water += right_max - heights[right]
    return water
```

Complexity: O(n) time, O(1) space.

1.10 Container With Most Water

Problem: Given heights of vertical lines, find two lines that form a container holding the most water.

Approach: Start with the widest container and always move the pointer at the shorter line inward, since that is the only way to potentially increase area.

```
def max_area(heights):
    left, right = 0, len(heights) - 1
    best = 0
    while left < right:
        width = right - left
        best = max(best, width * min(heights[left], heights[right]))
        if heights[left] < heights[right]:
            left += 1
        else:
            right -= 1
    return best
```

Complexity: O(n) time, O(1) space.

Pattern 2: Sliding Window

When to use it: You need to find a contiguous subarray/substring that optimizes some condition (max/min length, sum, count of distinct characters, etc.). Instead of recomputing from scratch for every start index, keep a window and slide it, adjusting a running state incrementally.

2.1 Maximum Sum Subarray of Size K

Problem: Find the maximum sum of any contiguous subarray of size K.

```
def max_sub_array_of_size_k(k, arr):
    window_sum, max_sum, start = 0.0, 0.0, 0
    for end in range(len(arr)):
        window_sum += arr[end]
        if end >= k - 1:
            max_sum = max(max_sum, window_sum)
            window_sum -= arr[start]
            start += 1
    return max_sum
```

Complexity: O(n) time, O(1) space.

2.2 Smallest Subarray with a Given Sum

Problem: Find the length of the smallest contiguous subarray whose sum is at least S.

```
def smallest_subarray_with_given_sum(s, arr):
    window_sum, min_length, start = 0, float('inf'), 0
    for end in range(len(arr)):
        window_sum += arr[end]
        while window_sum >= s:
            min_length = min(min_length, end - start + 1)
            window_sum -= arr[start]
            start += 1
    return 0 if min_length == float('inf') else min_length
```

Complexity: O(n) time, O(1) space.

2.3 Longest Substring with K Distinct Characters

```
def longest_substring_with_k_distinct(s, k):
    from collections import defaultdict
    char_freq = defaultdict(int)
    start, max_length = 0, 0
    for end in range(len(s)):
        char_freq[s[end]] += 1
        while len(char_freq) > k:
            char_freq[s[start]] -= 1
            if char_freq[s[start]] == 0:
                del char_freq[s[start]]
            start += 1
        max_length = max(max_length, end - start + 1)
    return max_length
```

Complexity: O(n) time, O(k) space.

2.4 Fruits into Baskets

Problem: You have two baskets; find the longest subarray containing at most 2 distinct fruit types. (Identical in structure to 2.3 with k = 2.)

```
def fruits_into_baskets(fruits):
    from collections import defaultdict
    basket = defaultdict(int)
    start, max_length = 0, 0
    for end in range(len(fruits)):
        basket[fruits[end]] += 1
        while len(basket) > 2:
            basket[fruits[start]] -= 1
            if basket[fruits[start]] == 0:
                del basket[fruits[start]]
            start += 1
        max_length = max(max_length, end - start + 1)
    return max_length
```

Complexity: O(n) time, O(1) space (at most 3 fruit keys at a time).

2.5 Longest Substring Without Repeating Characters

```
def longest_substring_without_repetition(s):
    char_index = {}
    start, max_length = 0, 0
    for end in range(len(s)):
        c = s[end]
        if c in char_index and char_index[c] >= start:
            start = char_index[c] + 1
        char_index[c] = end
        max_length = max(max_length, end - start + 1)
    return max_length
```

Complexity: O(n) time, O(min(n, alphabet)) space.

2.6 Longest Substring with Same Letters after Replacement

Problem: Given a string and a number K, find the length of the longest substring having the same letter after replacing at most K characters.

Approach: Track the frequency of the most common character in the window. The window is valid as long as `window_length - max_repeat_count <= k`.

```
def length_of_longest_substring(s, k):
    from collections import defaultdict
    freq = defaultdict(int)
    start, max_length, max_repeat_count = 0, 0, 0
    for end in range(len(s)):
        freq[s[end]] += 1
        max_repeat_count = max(max_repeat_count, freq[s[end]])
        if (end - start + 1) - max_repeat_count > k:
            freq[s[start]] -= 1
            start += 1
        max_length = max(max_length, end - start + 1)
    return max_length
```

Complexity: O(n) time, O(alphabet) space.

2.7 Longest Subarray with Ones after Replacement

Problem: Given a binary array, find the longest subarray of 1s after flipping at most K zeros.

```
def length_of_longest_substring_ones(arr, k):
    start, max_length, max_ones_count = 0, 0, 0
    for end in range(len(arr)):
        if arr[end] == 1:
            max_ones_count += 1
        if (end - start + 1) - max_ones_count > k:
            if arr[start] == 1:
                max_ones_count -= 1
            start += 1
        max_length = max(max_length, end - start + 1)
    return max_length
```

Complexity: $O(n)$ time, $O(1)$ space.

2.8 Permutation in a String

Problem: Given a string and a pattern, check if the string contains a permutation of the pattern.

```
def find_permutation(str1, pattern):
    from collections import Counter
    char_freq = Counter(pattern)
    matched = 0
    start = 0
    for end in range(len(str1)):
        right_char = str1[end]
        if right_char in char_freq:
            char_freq[right_char] -= 1
            if char_freq[right_char] == 0:
                matched += 1
        if matched == len(char_freq):
            return True
        if end - start + 1 >= len(pattern):
            left_char = str1[start]
            start += 1
            if left_char in char_freq:
                if char_freq[left_char] == 0:
                    matched -= 1
                char_freq[left_char] += 1
    return False
```

Complexity: $O(n + m)$ time, $O(m)$ space.

2.9 String Anagrams (Find All Anagram Start Indices)

```
def find_string_anagrams(str1, pattern):
    from collections import Counter
    char_freq = Counter(pattern)
    result, matched, start = [], 0, 0
    for end in range(len(str1)):
        right_char = str1[end]
        if right_char in char_freq:
            char_freq[right_char] -= 1
            if char_freq[right_char] == 0:
                matched += 1
        if matched == len(char_freq):
            result.append(start)
        if end - start + 1 >= len(pattern):
            left_char = str1[start]
            start += 1
            if left_char in char_freq:
                if char_freq[left_char] == 0:
                    matched -= 1
                char_freq[left_char] += 1
    return result
```

Complexity: $O(n + m)$ time, $O(m)$ space.

2.10 Minimum Window Substring

Problem: Find the smallest substring of `s` that contains all characters of `t` (with multiplicity).

```
def min_window(s, t):
    from collections import Counter
    if not s or not t:
        return ""
    need = Counter(t)
    missing = len(t)
    start, best_start, best_len = 0, 0, float('inf')
    for end, char in enumerate(s, 1):
        if need[char] > 0:
            missing -= 1
            need[char] -= 1
        if missing == 0:
            while need[s[start]] < 0:
                need[s[start]] += 1
                start += 1
            if end - start < best_len:
                best_start, best_len = start, end - start
            need[s[start]] += 1
            missing += 1
            start += 1
    return "" if best_len == float('inf') else s[best_start:best_start + best_len]
```

Complexity: $O(n + m)$ time, $O(m)$ space.

Pattern 3: Fast & Slow Pointers

When to use it: Problems on linked lists or sequences where you need to detect cycles, find the middle, or find a specific offset from the end, without knowing the length in advance. Two pointers move at different speeds (usually 1x and 2x).

3.1 LinkedList Cycle Detection

```
class ListNode:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

def has_cycle(head):
    slow, fast = head, head
    while fast is not None and fast.next is not None:
        fast = fast.next.next
        slow = slow.next
        if slow == fast:
            return True
    return False
```

Complexity: $O(n)$ time, $O(1)$ space.

3.2 Start of LinkedList Cycle

Approach: After detecting a cycle (3.1), reset one pointer to the head and move both one step at a time; they meet at the cycle's start.

```
def find_cycle_start(head):
    slow, fast = head, head
    while True:
        if fast is None or fast.next is None:
            return None
        fast = fast.next.next
        slow = slow.next
        if slow == fast:
            break
    pointer = head
    while pointer != slow:
        pointer = pointer.next
        slow = slow.next
    return pointer
```

Complexity: $O(n)$ time, $O(1)$ space.

3.3 Happy Number

Problem: Determine if repeatedly summing the squares of a number's digits eventually reaches 1.

Approach: Treat the sequence of sums as an implicit linked list; a non-happy number falls into a cycle.

```

def find_square_sum(num):
    total = 0
    while num > 0:
        digit = num % 10
        total += digit * digit
        num //= 10
    return total

def is_happy_number(num):
    slow, fast = num, num
    while True:
        slow = find_square_sum(slow)
        fast = find_square_sum(find_square_sum(fast))
        if slow == fast:
            break
    return slow == 1

```

Complexity: $O(\log n)$ per squaring step; overall bounded and effectively $O(1)$ amortized for typical inputs.

3.4 Middle of the LinkedList

```

def find_middle(head):
    slow, fast = head, head
    while fast is not None and fast.next is not None:
        fast = fast.next.next
        slow = slow.next
    return slow

```

Complexity: $O(n)$ time, $O(1)$ space.

3.5 Palindrome LinkedList

Approach: Find the middle, reverse the second half, compare both halves.

```

def is_palindromic_linked_list(head):
    if head is None or head.next is None:
        return True
    slow, fast = head, head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

    def reverse(node):
        prev = None
        while node:
            nxt = node.next
            node.next = prev
            prev = node
            node = nxt
        return prev

    second_half = reverse(slow)
    copy_half = second_half
    first_half = head
    matched = True
    while first_half and second_half:
        if first_half.value != second_half.value:
            matched = False
            break
        first_half = first_half.next
        second_half = second_half.next
    reverse(copy_half)
    return matched

```

Complexity: $O(n)$ time, $O(1)$ space.

3.6 Rearrange a LinkedList (Odd-Even Interleave Reversal)

Problem: Rearrange $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ into $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$.

Approach: Find the middle, reverse the second half, then weave the two halves together.

```
def reorder(head):
    if head is None or head.next is None:
        return
    slow, fast = head, head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

    prev, node = None, slow
    while node:
        nxt = node.next
        node.next = prev
        prev = node
        node = nxt
    second = prev

    first = head
    while first is not None and second is not None:
        temp = first.next
        first.next = second
        first = temp
        temp = second.next
        second.next = first
        second = temp
```

Complexity: $O(n)$ time, $O(1)$ space.

3.7 Cycle in a Circular Array

Problem: Given an array of non-zero integers where each element represents a jump (positive = forward, negative = backward), detect if there's a cycle of length > 1 with all same-direction moves.

```
def circular_array_loop_exists(arr):
    n = len(arr)

    def next_index(i):
        return (i + arr[i]) % n

    for i in range(n):
        is_forward = arr[i] > 0
        slow, fast = i, i
        while True:
            slow = next_index(slow)
            if is_forward != (arr[slow] > 0):
                break
            fast = next_index(fast)
            if is_forward != (arr[fast] > 0):
                break
            fast = next_index(fast)
            if is_forward != (arr[fast] > 0):
                break
            if slow == fast:
                if slow == next_index(slow):
                    break
                return True
        return False
```

Complexity: $O(n^2)$ worst case, $O(1)$ space.

3.8 Find the Duplicate Number

Problem: Given $n+1$ integers in range $[1, n]$, find the one duplicate (each number can repeat).

Approach: Treat array values as pointers ($\text{arr}[i]$ points to index $\text{arr}[i]$); the duplicate creates a cycle, found the same way as LinkedList Cycle.

```
def find_duplicate(arr):
    slow, fast = arr[0], arr[arr[0]]
    while slow != fast:
        slow = arr[slow]
        fast = arr[arr[fast]]
    fast = 0
    while slow != fast:
        slow = arr[slow]
        fast = arr[fast]
    return slow
```

Complexity: $O(n)$ time, $O(1)$ space.

3.9 Remove Nth Node From End of List

Approach: Move **fast** n steps ahead first, then move both together; when **fast** hits the end, **slow** is just before the node to remove.

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    slow = fast = dummy
    for _ in range(n):
        fast = fast.next
    while fast.next:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

Complexity: $O(n)$ time, $O(1)$ space.

3.10 Reorder List (LeetCode variant using fast/slow explicitly)

Problem: Same objective as 3.6, phrased as a standalone LeetCode-style problem — reinforces recognizing the split -> reverse -> merge sequence as a reusable sub-routine on any list-reordering task.

```
def reorder_list(head):
    if not head:
        return
    slow, fast = head, head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    second = slow.next
    slow.next = None
    prev = None
    while second:
        nxt = second.next
        second.next = prev
        prev = second
        second = nxt
    first, second = head, prev
    while second:
        tmp1, tmp2 = first.next, second.next
        first.next = second
        second.next = tmp1
        first, second = tmp1, tmp2
```

Complexity: $O(n)$ time, $O(1)$ space.

Pattern 4: Merge Intervals

When to use it: Any problem dealing with overlapping ranges — scheduling, merging, or finding gaps. The core trick is almost always: **sort by start time**, then walk through comparing each interval to the last one you've kept.

4.1 Merge Intervals

```
def merge(intervals):
    if not intervals:
        return []
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for interval in intervals[1:]:
        last = merged[-1]
        if interval[0] <= last[1]:
            last[1] = max(last[1], interval[1])
        else:
            merged.append(interval)
    return merged
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

4.2 Insert Interval

```
def insert_interval(intervals, new_interval):
    merged = []
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < new_interval[0]:
        merged.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] <= new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1
    merged.append(new_interval)
    while i < n:
        merged.append(intervals[i])
        i += 1
    return merged
```

Complexity: $O(n)$ time, $O(n)$ space.

4.3 Intervals Intersection

Problem: Given two lists of sorted, disjoint intervals, find their intersection.

```
def intervals_intersection(a, b):
    result = []
    i, j = 0, 0
    while i < len(a) and j < len(b):
        start = max(a[i][0], b[j][0])
        end = min(a[i][1], b[j][1])
        if start <= end:
            result.append([start, end])
        if a[i][1] < b[j][1]:
            i += 1
        else:
            j += 1
    return result
```

Complexity: $O(n + m)$ time, $O(n + m)$ space.

4.4 Conflicting Appointments (Can Attend All Meetings)

```
def can_attend_all_meetings(intervals):
    intervals.sort(key=lambda x: x[0])
    for i in range(1, len(intervals)):
        if intervals[i][0] < intervals[i - 1][1]:
            return False
    return True
```

Complexity: $O(n \log n)$ time, $O(1)$ extra space.

4.5 Minimum Meeting Rooms

Approach: Use two heaps (or sorted start/end arrays) — push start times, and whenever a meeting starts, pop any meetings that have already ended from a min-heap of end times.

```
import heapq

def min_meeting_rooms(intervals):
    if not intervals:
        return 0
    intervals.sort(key=lambda x: x[0])
    min_heap = []
    for interval in intervals:
        if min_heap and min_heap[0] <= interval[0]:
            heapq.heapreplace(min_heap, interval[1])
        else:
            heapq.heappush(min_heap, interval[1])
    return len(min_heap)
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

4.6 Maximum CPU Load

Problem: Given jobs with start, end, and CPU load, find the maximum CPU load at any point in time.

```
import heapq

def find_max_cpu_load(jobs):
    jobs.sort(key=lambda x: x[0])
    max_load, current_load = 0, 0
    min_heap = [] # (end, load)
    for job in jobs:
        start, end, load = job
        while min_heap and min_heap[0][0] <= start:
            current_load -= heapq.heappop(min_heap)[1]
        heapq.heappush(min_heap, (end, load))
        current_load += load
        max_load = max(max_load, current_load)
    return max_load
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

4.7 Employee Free Time

Problem: Given schedules for multiple employees (each a list of sorted, non-overlapping intervals), find the common free time across all employees.

```
def employee_free_time(schedule):
    all_intervals = sorted((iv for employee in schedule for iv in employee), key=lambda x: x[0])
    merged = [all_intervals[0]]
    for iv in all_intervals[1:]:
        last = merged[-1]
        if iv[0] <= last[1]:
            last[1] = max(last[1], iv[1])
        else:
            merged.append(iv)
    free_time = []
    for i in range(1, len(merged)):
        free_time.append([merged[i - 1][1], merged[i][0]])
    return free_time
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

4.8 Non-overlapping Intervals

Problem: Find the minimum number of intervals to remove to make the rest non-overlapping.

Approach: Sort by end time; greedily keep the interval that ends earliest whenever there's a conflict.

```
def erase_overlap_intervals(intervals):
    if not intervals:
        return 0
    intervals.sort(key=lambda x: x[1])
    count, prev_end = 0, intervals[0][1]
    for start, end in intervals[1:]:
        if start < prev_end:
            count += 1
        else:
            prev_end = end
    return count
```

Complexity: $O(n \log n)$ time, $O(1)$ extra space.

4.9 Interval List Intersections (Merge and Count Overlaps)

Problem: Given a list of intervals possibly with overlaps and duplicates, merge them and report how many original intervals contributed to the busiest merged interval.

```
def merge_and_count(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = []
    contributors = []
    for interval in intervals:
        if merged and interval[0] <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], interval[1])
            contributors[-1] += 1
        else:
            merged.append(list(interval))
            contributors.append(1)
    return merged, contributors
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

4.10 Partition Labels

Problem: Partition a string into as many parts as possible so that each letter appears in at most one part.

Approach: This is a “merge intervals” problem in disguise — the interval for each character spans its first to last occurrence.

```
def partition_labels(s):
    last_occurrence = {c: i for i, c in enumerate(s)}
    result = []
    start = end = 0
    for i, c in enumerate(s):
        end = max(end, last_occurrence[c])
        if i == end:
            result.append(end - start + 1)
            start = i + 1
    return result
```

Complexity: $O(n)$ time, $O(1)$ extra space (fixed alphabet).

Pattern 5: Cyclic Sort

When to use it: You have an array containing numbers in a known, contiguous range (e.g., `1..n` or `0..n-1`), and you need to find missing/duplicate numbers in $O(n)$ time and $O(1)$ space. The trick: place every number at its “correct” index by swapping.

5.1 Cyclic Sort

```
def cyclic_sort(nums):
    i = 0
    while i < len(nums):
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    return nums
```

Complexity: $O(n)$ time, $O(1)$ space.

5.2 Find the Missing Number

```
def find_missing_number(nums):
    i, n = 0, len(nums)
    while i < n:
        correct = nums[i]
        if correct < n and nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    for i in range(n):
        if nums[i] != i:
            return i
    return n
```

Complexity: $O(n)$ time, $O(1)$ space.

5.3 Find All Missing Numbers

```
def find_all_missing_numbers(nums):
    i, n = 0, len(nums)
    while i < n:
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    missing = []
    for i in range(n):
        if nums[i] != i + 1:
            missing.append(i + 1)
    return missing
```

Complexity: $O(n)$ time, $O(1)$ extra space.

5.4 Find the Duplicate Number (1..n-1 range)

```
def find_duplicate_cyclic(nums):
    i = 0
    while i < len(nums):
        if nums[i] != i + 1:
            correct = nums[i] - 1
            if nums[i] != nums[correct]:
                nums[i], nums[correct] = nums[correct], nums[i]
            else:
                return nums[i]
        else:
            i += 1
    return -1
```

Complexity: O(n) time, O(1) space.

5.5 Find All Duplicate Numbers

```
def find_all_duplicates(nums):
    i = 0
    while i < len(nums):
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    duplicates = []
    for i in range(len(nums)):
        if nums[i] != i + 1:
            duplicates.append(nums[i])
    return duplicates
```

Complexity: O(n) time, O(1) extra space.

5.6 Find the Corrupt Pair (One Missing, One Duplicate)

```
def find_corrupt_pair(nums):
    i = 0
    while i < len(nums):
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    for i in range(len(nums)):
        if nums[i] != i + 1:
            return [nums[i], i + 1]
    return [-1, -1]
```

Complexity: O(n) time, O(1) space.

5.7 First Missing Positive

Problem: Given an unsorted array, find the smallest missing positive integer.

```
def first_missing_positive(nums):
    i, n = 0, len(nums)
    while i < n:
        correct = nums[i] - 1
        if 0 < nums[i] <= n and nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1
```

Complexity: $O(n)$ time, $O(1)$ space.

5.8 Kth Missing Positive Number

Problem: Given a sorted array of positives and K , find the K th positive number that is missing.

Approach: Not a direct swap problem, but shares the “expected index vs actual value” mindset from cyclic sort. Binary search on the number of missing elements before each index.

```
def find_kth_positive(nums, k):
    left, right = 0, len(nums)
    while left < right:
        mid = (left + right) // 2
        missing_before = nums[mid] - (mid + 1)
        if missing_before < k:
            left = mid + 1
        else:
            right = mid
    return left + k
```

Complexity: $O(\log n)$ time, $O(1)$ space.

5.9 Set Mismatch

Problem: A set of numbers $1..n$ had one number replaced by a duplicate of another. Find the duplicate and the missing number.

```
def find_error_nums(nums):
    i = 0
    while i < len(nums):
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    for i in range(len(nums)):
        if nums[i] != i + 1:
            return [nums[i], i + 1]
    return []
```

Complexity: $O(n)$ time, $O(1)$ space. (Same routine as 5.6, applied to LeetCode’s phrasing.)

5.10 Smallest Missing Positive After Replacing Negatives

Problem: Given an array with negatives, zeros, and positives, first neutralize negatives/zeros/out-of-range values, then find the first missing positive.

```
def first_missing_positive_general(nums):
    n = len(nums)
    for i in range(n):
        if nums[i] <= 0 or nums[i] > n:
            nums[i] = n + 1
    i = 0
    while i < n:
        correct = nums[i] - 1
        if nums[i] <= n and nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1
```

Complexity: $O(n)$ time, $O(1)$ space.

Pattern 6: In-place Reversal of a LinkedList

When to use it: You need to reverse a linked list (or parts of it) without using $O(n)$ extra space. The core routine: walk the list keeping `prev`, `curr`, `next` pointers, and flip the `next` pointer of each node.

```
class ListNode:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

6.1 Reverse a LinkedList

```
def reverse(head):
    prev = None
    while head is not None:
        nxt = head.next
        head.next = prev
        prev = head
        head = nxt
    return prev
```

Complexity: $O(n)$ time, $O(1)$ space.

6.2 Reverse a Sub-list (Between Positions p and q)

```
def reverse_sub_list(head, p, q):
    if p == q:
        return head
    dummy = ListNode(0, head)
    prev = dummy
    for _ in range(p - 1):
        prev = prev.next
    curr = prev.next
    for _ in range(q - p):
        nxt = curr.next
        curr.next = nxt.next
        nxt.next = prev.next
        prev.next = nxt
    return dummy.next
```

Complexity: $O(q)$ time, $O(1)$ space.

6.3 Reverse Every K-element Sub-list

```
def reverse_every_k_elements(head, k):
    dummy = ListNode(0, head)
    prev_group_end = dummy
    curr = head
    while curr:
        group_start = curr
        prev, count = None, 0
        while curr and count < k:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
            count += 1
        old_next = prev_group_end.next
        prev_group_end.next = prev
        old_next.next = curr
        prev_group_end = old_next
    return dummy.next
```

Complexity: $O(n)$ time, $O(1)$ space.

6.4 Reverse Alternating K-element Sub-list

```
def reverse_alternating_k_elements(head, k):
    dummy = ListNode(0, head)
    prev_group_end = dummy
    curr = head
    while curr:
        group_start = curr
        prev, count = None, 0
        while curr and count < k:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
            count += 1

        old_next = prev_group_end.next
        prev_group_end.next = prev
        old_next.next = curr
        prev_group_end = old_next
        # skip the next k nodes without reversing
        count = 0
        while curr and count < k:
            prev_group_end = curr
            curr = curr.next
            count += 1
    return dummy.next
```

Complexity: O(n) time, O(1) space.

6.5 Rotate a LinkedList

Problem: Rotate the list to the right by K places.

```
def rotate_right(head, k):
    if not head or not head.next or k == 0:
        return head
    length, tail = 1, head
    while tail.next:
        tail = tail.next
        length += 1
    k %= length
    if k == 0:
        return head
    tail.next = head # make circular
    steps_to_new_tail = length - k
    new_tail = head
    for _ in range(steps_to_new_tail - 1):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
    return new_head
```

Complexity: O(n) time, O(1) space.

6.6 Swap Nodes in Pairs

```
def swap_pairs(head):
    dummy = ListNode(0, head)
    prev = dummy
    while prev.next and prev.next.next:
        first = prev.next
        second = first.next
        first.next = second.next
        second.next = first
        prev.next = second
        prev = first
    return dummy.next
```

Complexity: O(n) time, O(1) space.

6.7 Reverse Nodes in Even Length Groups

Problem: Nodes are grouped in increasing sizes 1, 2, 3, ... (last group may be shorter). Reverse only the groups with an even number of nodes.

```
def reverse_even_length_groups(head):
    prev, curr, group = head, head.next, 2
    while curr:
        node, num_nodes = prev, 0
        for _ in range(group):
            if not curr:
                break
            num_nodes += 1
            node = curr
            curr = curr.next
        if num_nodes % 2 == 0:
            reverse_curr = curr
            node = prev.next
            for _ in range(num_nodes):
                nxt = node.next
                node.next = reverse_curr
                reverse_curr = node
                node = nxt
            temp = prev.next
            prev.next = reverse_curr
            prev = temp
        else:
            prev = node
        group += 1
    return head
```

Complexity: O(n) time, O(1) space.

6.8 Palindrome Check via In-place Reversal

Problem: Same goal as 3.5, but this version explicitly reverses the first half in place (rather than just the second half) to double-check the result, reinforcing the reversal routine as a composable building block.

```
def is_palindrome_via_reversal(head):
    slow, fast = head, head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    prev = None
    while slow:
        nxt = slow.next
        slow.next = prev
        prev = slow
        slow = nxt
    left, right = head, prev
    is_palin = True
    while right:
        if left.value != right.value:
            is_palin = False
            break
        left, right = left.next, right.next
    return is_palin
```

Complexity: O(n) time, O(1) space.

6.9 Add Two Numbers as LinkedLists

Problem: Two numbers represented as linked lists (digits in reverse order); return their sum as a linked list.

```
def add_two_numbers(l1, l2):
    dummy = ListNode(0)
    curr = dummy
    carry = 0
    while l1 or l2 or carry:
        v1 = l1.value if l1 else 0
        v2 = l2.value if l2 else 0
        total = v1 + v2 + carry
        carry = total // 10
        curr.next = ListNode(total % 10)
        curr = curr.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next
```

Complexity: $O(\max(n, m))$ time, $O(\max(n, m))$ space for the result.

6.10 Split LinkedList in Parts

Problem: Split a linked list into K consecutive parts as evenly as possible.

```
def split_list_to_parts(head, k):
    length = 0
    node = head
    while node:
        length += 1
        node = node.next
    base_size, extra = divmod(length, k)
    result = []
    curr = head
    for i in range(k):
        part_size = base_size + (1 if i < extra else 0)
        if part_size == 0:
            result.append(None)
            continue
        part_head = curr
        for _ in range(part_size - 1):
            curr = curr.next
        nxt = curr.next
        curr.next = None
        curr = nxt
        result.append(part_head)
    return result
```

Complexity: $O(n)$ time, $O(k)$ space for the output list.

Pattern 7: Tree Breadth-First Search (BFS)

When to use it: Any tree problem that requires level-by-level processing — level order traversal, finding shortest depth, connecting nodes at the same level, etc. Use a queue and process one full level at a time.

```
class TreeNode:
    def __init__(self, val, left=None, right=None, next=None):
        self.val = val
        self.left = left
        self.right = right
        self.next = next # used for "connect" style problems
```

7.1 Binary Tree Level Order Traversal

```
from collections import deque

def level_order(root):
    result = []
    if not root:
        return result
    queue = deque([root])
    while queue:
        level_size = len(queue)
        level = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

Complexity: $O(n)$ time, $O(n)$ space.

7.2 Reverse Level Order Traversal

```
from collections import deque

def reverse_level_order(root):
    result = []
    if not root:
        return result
    queue = deque([root])
    while queue:
        level_size = len(queue)
        level = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.insert(0, level)
    return result
```

Complexity: $O(n)$ time, $O(n)$ space.

7.3 Zigzag Traversal

```

from collections import deque

def zigzag_level_order(root):
    result = []
    if not root:
        return result
    queue = deque([root])
    left_to_right = True
    while queue:
        level_size = len(queue)
        level = deque()
        for _ in range(level_size):
            node = queue.popleft()
            if left_to_right:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(list(level))
        left_to_right = not left_to_right
    return result

```

Complexity: $O(n)$ time, $O(n)$ space.

7.4 Level Averages in Binary Tree

```

from collections import deque

def find_level_averages(root):
    result = []
    if not root:
        return result
    queue = deque([root])
    while queue:
        level_size = len(queue)
        level_sum = 0
        for _ in range(level_size):
            node = queue.popleft()
            level_sum += node.val
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level_sum / level_size)
    return result

```

Complexity: $O(n)$ time, $O(n)$ space.

7.5 Minimum Depth of Binary Tree

Approach: BFS is ideal here (rather than DFS) because the first leaf you encounter level-by-level guarantees the minimum depth.

```

from collections import deque

def find_min_depth(root):
    if not root:
        return 0
    queue = deque([root])
    depth = 1
    while queue:
        level_size = len(queue)
        for _ in range(level_size):
            node = queue.popleft()
            if not node.left and not node.right:
                return depth
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        depth += 1
    return depth

```

Complexity: $O(n)$ time, $O(n)$ space.

7.6 Maximum Depth of Binary Tree (BFS)

```

from collections import deque

def find_max_depth(root):
    if not root:
        return 0
    queue = deque([root])
    depth = 0
    while queue:
        depth += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return depth

```

Complexity: $O(n)$ time, $O(n)$ space.

7.7 Level Order Successor

Problem: Given a node's value, find the node that comes right after it in level order.

```

from collections import deque

def find_successor(root, key):
    if not root:
        return None
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
        if node.val == key:
            break
    return queue[0] if queue else None

```

Complexity: $O(n)$ time, $O(n)$ space.

7.8 Connect Level Order Siblings

Problem: Connect each node to its next right node at the same level (the last node in a level points to `None`).

```
from collections import deque

def connect_all_siblings(root):
    if not root:
        return
    queue = deque([root])
    while queue:
        level_size = len(queue)
        prev = None
        for _ in range(level_size):
            node = queue.popleft()
            if prev:
                prev.next = node
            prev = node
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
```

Complexity: $O(n)$ time, $O(n)$ space.

7.9 Right View of Binary Tree

```
from collections import deque

def right_view(root):
    result = []
    if not root:
        return result
    queue = deque([root])
    while queue:
        level_size = len(queue)
        for i in range(level_size):
            node = queue.popleft()
            if i == level_size - 1:
                result.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return result
```

Complexity: $O(n)$ time, $O(n)$ space.

7.10 Binary Tree Level Order Traversal II (Bottom-up)

Problem: Same as 7.1 but return levels from bottom to top (identical technique to 7.2, included here since it is a very common standalone LeetCode phrasing).

```
from collections import deque

def level_order_bottom(root):
    result = []
    if not root:
        return result
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result[::-1]
```

Complexity: $O(n)$ time, $O(n)$ space.

Pattern 8: Tree Depth-First Search (DFS)

When to use it: Problems involving paths from root to leaf, path sums, or anything requiring exploration of one branch fully before backtracking. Usually implemented recursively, carrying state (running sum, path list) down the call stack.

8.1 Binary Tree Path Sum

Problem: Does any root-to-leaf path sum equal a given value?

```
def has_path_sum(root, target_sum):
    if not root:
        return False
    if not root.left and not root.right:
        return root.val == target_sum
    remaining = target_sum - root.val
    return has_path_sum(root.left, remaining) or has_path_sum(root.right, remaining)
```

Complexity: $O(n)$ time, $O(h)$ space (recursion depth).

8.2 All Paths for a Sum

```
def find_paths(root, target_sum):
    all_paths = []
    find_paths_recursive(root, target_sum, [], all_paths)
    return all_paths

def find_paths_recursive(node, target_sum, current_path, all_paths):
    if not node:
        return
    current_path.append(node.val)
    if not node.left and not node.right and node.val == target_sum:
        all_paths.append(list(current_path))
    else:
        find_paths_recursive(node.left, target_sum - node.val, current_path, all_paths)
        find_paths_recursive(node.right, target_sum - node.val, current_path, all_paths)
    current_path.pop()
```

Complexity: $O(n^2)$ worst case (copying paths), $O(h)$ recursion space.

8.3 Sum of Path Numbers

Problem: Each root-to-leaf path forms a number (digits = node values). Return the sum of all such numbers.

```
def find_sum_of_path_numbers(root):
    return find_root_to_leaf_sum(root, 0)

def find_root_to_leaf_sum(node, path_sum):
    if not node:
        return 0
    path_sum = path_sum * 10 + node.val
    if not node.left and not node.right:
        return path_sum
    return find_root_to_leaf_sum(node.left, path_sum) + find_root_to_leaf_sum(node.right, path_sum)
```

Complexity: $O(n)$ time, $O(h)$ space.

8.4 Path With Given Sequence

Problem: Check if the tree has a root-to-leaf path matching a given sequence exactly.

```
def find_path(root, sequence):
    if not root:
        return len(sequence) == 0
    return find_path_recursive(root, sequence, 0)

def find_path_recursive(node, sequence, index):
    if not node or index >= len(sequence) or node.val != sequence[index]:
        return False
    if not node.left and not node.right and index == len(sequence) - 1:
        return True
    return (find_path_recursive(node.left, sequence, index + 1) or
            find_path_recursive(node.right, sequence, index + 1))
```

Complexity: $O(n)$ time, $O(h)$ space.

8.5 Count Paths for a Sum

Problem: Count all paths (not necessarily root-to-leaf) that sum to a target.

Approach: DFS while maintaining a running prefix-sum count map, similar to the subarray-sum-equals-K trick.

```
from collections import defaultdict

def count_paths(root, target_sum):
    return count_paths_recursive(root, target_sum, [], 0)

def count_paths_recursive(node, target_sum, current_path, path_count):
    if not node:
        return path_count
    current_path.append(node.val)
    path_sum = 0
    for i in range(len(current_path) - 1, -1, -1):
        path_sum += current_path[i]
        if path_sum == target_sum:
            path_count += 1
    path_count = count_paths_recursive(node.left, target_sum, current_path, path_count)
    path_count = count_paths_recursive(node.right, target_sum, current_path, path_count)
    current_path.pop()
    return path_count
```

Complexity: $O(n \log n)$ average ($O(n^2)$ worst case for skewed trees), $O(h)$ space.

8.6 Diameter of Binary Tree

Problem: Find the length of the longest path between any two nodes in the tree.

```
def diameter_of_binary_tree(root):
    diameter = 0

    def height(node):
        nonlocal diameter
        if not node:
            return 0
        left = height(node.left)
        right = height(node.right)
        diameter = max(diameter, left + right)
        return 1 + max(left, right)

    height(root)
    return diameter
```

Complexity: $O(n)$ time, $O(h)$ space.

8.7 Lowest Common Ancestor

```
def lowest_common_ancestor(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left and right:
        return root
    return left if left else right
```

Complexity: $O(n)$ time, $O(h)$ space.

8.8 Path Sum III (Count Subpaths, Not Necessarily Root-Rooted)

Problem: Count the number of downward paths (not necessarily starting at root) that sum to a target.

Approach: Use a prefix-sum hash map passed through DFS, similar in spirit to 8.5 but $O(n)$ via hashing instead of scanning the path array.

```
from collections import defaultdict

def path_sum_iii(root, target_sum):
    prefix_counts = defaultdict(int)
    prefix_counts[0] = 1

    def dfs(node, curr_sum):
        if not node:
            return 0
        curr_sum += node.val
        count = prefix_counts[curr_sum - target_sum]
        prefix_counts[curr_sum] += 1
        count += dfs(node.left, curr_sum)
        count += dfs(node.right, curr_sum)
        prefix_counts[curr_sum] -= 1
        return count

    return dfs(root, 0)
```

Complexity: $O(n)$ time, $O(n)$ space.

8.9 Binary Tree Maximum Path Sum

Problem: Find the maximum path sum between any two nodes (path need not pass through the root).

```
def max_path_sum(root):
    best = float('-inf')

    def dfs(node):
        nonlocal best
        if not node:
            return 0
        left = max(dfs(node.left), 0)
        right = max(dfs(node.right), 0)
        best = max(best, node.val + left + right)
        return node.val + max(left, right)

    dfs(root)
    return best
```

Complexity: $O(n)$ time, $O(h)$ space.

8.10 Serialize and Deserialize Binary Tree

Problem: Convert a tree to a string and back, preserving structure.

Approach: Pre-order DFS, using a sentinel for `None` children.

```

def serialize(root):
    result = []

    def dfs(node):
        if not node:
            result.append("N")
            return
        result.append(str(node.val))
        dfs(node.left)
        dfs(node.right)

    dfs(root)
    return ",".join(result)

def deserialize(data):
    values = iter(data.split(","))

    def dfs():
        val = next(values)
        if val == "N":
            return None
        node = TreeNode(int(val))
        node.left = dfs()
        node.right = dfs()
        return node

    return dfs()

```

Complexity: $O(n)$ time, $O(n)$ space.

Pattern 9: Two Heaps

When to use it: Problems that need to track two halves of a dataset (e.g., smaller half / larger half for a running median), or need simultaneous access to a min and a max under changing data. Maintain a max-heap for the lower half and a min-heap for the upper half.

(Python's `heapq` is a min-heap; simulate a max-heap by pushing negated values.)

9.1 Find the Median of a Number Stream

```
import heapq

class MedianFinder:
    def __init__(self):
        self.max_heap = [] # lower half, negated
        self.min_heap = [] # upper half

    def insert_num(self, num):
        if not self.max_heap or -self.max_heap[0] >= num:
            heapq.heappush(self.max_heap, -num)
        else:
            heapq.heappush(self.min_heap, num)
        if len(self.max_heap) > len(self.min_heap) + 1:
            heapq.heappush(self.min_heap, -heapq.heappop(self.max_heap))
        elif len(self.min_heap) > len(self.max_heap):
            heapq.heappush(self.max_heap, -heapq.heappop(self.min_heap))

    def find_median(self):
        if len(self.max_heap) == len(self.min_heap):
            return (-self.max_heap[0] + self.min_heap[0]) / 2.0
        return -self.max_heap[0]
```

Complexity: $O(\log n)$ per insert, $O(1)$ per median query.

9.2 Sliding Window Median

```

import heapq
from collections import defaultdict

def median_sliding_window(nums, k):
    def add_num(num, max_heap, min_heap):
        if not max_heap or -max_heap[0] >= num:
            heapq.heappush(max_heap, -num)
        else:
            heapq.heappush(min_heap, num)

    def rebalance(max_heap, min_heap):
        if len(max_heap) > len(min_heap) + 1:
            heapq.heappush(min_heap, -heapq.heappop(max_heap))
        elif len(max_heap) < len(min_heap):
            heapq.heappush(max_heap, -heapq.heappop(min_heap))

    def get_median(max_heap, min_heap, k):
        if k % 2 == 0:
            return (-max_heap[0] + min_heap[0]) / 2.0
        return -max_heap[0] * 1.0

    result = []
    max_heap, min_heap = [], []
    hash_map = defaultdict(int)
    for i in range(len(nums)):
        add_num(nums[i], max_heap, min_heap)
        rebalance(max_heap, min_heap)
        if i >= k - 1:
            result.append(get_median(max_heap, min_heap, k))
            out_num = nums[i - k + 1]
            hash_map[out_num] += 1
            if out_num <= -max_heap[0]:
                if out_num == -max_heap[0]:
                    heapq.heappop(max_heap)
                else:
                    max_heap.remove(-out_num)
                    heapq.heapify(max_heap)
            else:
                if out_num == min_heap[0]:
                    heapq.heappop(min_heap)
                else:
                    min_heap.remove(out_num)
                    heapq.heapify(min_heap)
            rebalance(max_heap, min_heap)
    return result

```

Complexity: $O(n \log k)$ time, $O(k)$ space.

9.3 Maximize Capital (IPO Problem)

Problem: Given projects with required capital and profit, and a starting capital, pick up to K projects to maximize final capital (must be able to afford a project to select it).

```

import heapq

def maximum_capital(capital, profits, num_projects, initial_capital):
    min_capital_heap = list(zip(capital, profits))
    heapq.heapify(min_capital_heap)
    max_profit_heap = []
    available_capital = initial_capital
    for _ in range(num_projects):
        while min_capital_heap and min_capital_heap[0][0] <= available_capital:
            c, p = heapq.heappop(min_capital_heap)
            heapq.heappush(max_profit_heap, -p)
        if not max_profit_heap:
            break
        available_capital += -heapq.heappop(max_profit_heap)
    return available_capital

```

Complexity: $O(n \log n)$ time, $O(n)$ space.

9.4 Next Interval

Problem: For each interval, find the interval that starts right after it ends (the “next interval”), or -1 if none.

```
import heapq

def find_next_interval(intervals):
    n = len(intervals)
    max_start_heap = [(-intervals[i][0], i) for i in range(n)]
    min_end_heap = [(intervals[i][1], i) for i in range(n)]
    heapq.heapify(max_start_heap)
    heapq.heapify(min_end_heap)
    result = [-1] * n
    for _ in range(n):
        top_end, top_idx = heapq.heappop(min_end_heap)
        if -max_start_heap[0][0] < top_end:
            continue
        temp = []
        while max_start_heap and -max_start_heap[0][0] >= top_end:
            temp.append(heapq.heappop(max_start_heap))
        result[top_idx] = temp[-1][1] if temp else -1
        for item in temp[:-1]:
            heapq.heappush(max_start_heap, item)
        if temp:
            heapq.heappush(max_start_heap, temp[-1])
    return result
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

9.5 Task Scheduler

Problem: Given tasks and a cooldown period `n` between identical tasks, find the minimum time to finish all tasks.

Approach: A max-heap of task frequencies combined with a cooldown queue — conceptually a “two structures” balancing act like two heaps.

```
import heapq
from collections import Counter, deque

def least_interval(tasks, n):
    counts = Counter(tasks)
    max_heap = [-c for c in counts.values()]
    heapq.heapify(max_heap)
    time = 0
    cooldown = deque()  # (available_time, count)
    while max_heap or cooldown:
        time += 1
        if max_heap:
            count = heapq.heappop(max_heap) + 1
            if count < 0:
                cooldown.append((time + n, count))
            if cooldown and cooldown[0][0] == time:
                heapq.heappush(max_heap, cooldown.popleft()[1])
    return time
```

Complexity: $O(m \log 26)$ time where m is number of tasks, $O(1)$ extra space.

9.6 Meeting Rooms II (Heap-based)

Problem: Same objective as 4.5, revisited to emphasize the two-heap-style bookkeeping (start-time heap conceptually paired with an end-time heap).

```
import heapq

def min_meeting_rooms_two_heap(intervals):
    start_heap = sorted(i[0] for i in intervals)
    end_heap = sorted(i[1] for i in intervals)
    rooms = available = 0
    e = 0
    for s in start_heap:
        while e < len(end_heap) and end_heap[e] <= s:
            available += 1
            e += 1
        if available > 0:
            available -= 1
        else:
            rooms += 1
    return rooms
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

9.7 Kth Largest Element in a Stream

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = nums
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)

    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

Complexity: $O(\log k)$ per insertion, $O(k)$ space.

9.8 Rearrange String K Distance Apart

Problem: Rearrange a string's characters so identical characters are at least K apart.

```
import heapq
from collections import Counter, deque

def rearrange_k_distance(s, k):
    if k <= 1:
        return s
    counts = Counter(s)
    max_heap = [(-freq, ch) for ch, freq in counts.items()]
    heapq.heapify(max_heap)
    result = []
    queue = deque()  # (freq, ch, available_index)
    while max_heap:
        freq, ch = heapq.heappop(max_heap)
        result.append(ch)
        queue.append((freq + 1, ch, len(result) + k - 1))
        if queue[0][2] == len(result):
            f, c, _ = queue.popleft()
            if -f > 0:
                heapq.heappush(max_heap, (f, c))
    return "".join(result) if len(result) == len(s) else ""
```

Complexity: $O(n \log \text{alphabet})$ time, $O(\text{alphabet})$ space.

9.9 Maximum Frequency Stack

Problem: Design a stack where `pop` removes the most frequent element (ties broken by most recently pushed).

```
from collections import defaultdict

class FreqStack:
    def __init__(self):
        self.freq = defaultdict(int)
        self.group = defaultdict(list)
        self.max_freq = 0

    def push(self, val):
        f = self.freq[val] + 1
        self.freq[val] = f
        self.max_freq = max(self.max_freq, f)
        self.group[f].append(val)

    def pop(self):
        val = self.group[self.max_freq].pop()
        self.freq[val] -= 1
        if not self.group[self.max_freq]:
            self.max_freq -= 1
        return val
```

Complexity: $O(1)$ amortized per operation, $O(n)$ space.

9.10 Trapping Rain Water II

Problem: Given a 2D elevation map, compute the total trapped water (3D version of pattern 1.9).

Approach: Push all boundary cells into a min-heap; repeatedly pop the lowest boundary cell and expand inward, tracking the highest wall seen so far.

```
import heapq

def trap_rain_water_2d(height_map):
    if not height_map or not height_map[0]:
        return 0
    m, n = len(height_map), len(height_map[0])
    visited = [[False] * n for _ in range(m)]
    heap = []
    for i in range(m):
        for j in range(n):
            if i == 0 or i == m - 1 or j == 0 or j == n - 1:
                heapq.heappush(heap, (height_map[i][j], i, j))
                visited[i][j] = True
    water = 0
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    while heap:
        height, x, y = heapq.heappop(heap)
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < m and 0 <= ny < n and not visited[nx][ny]:
                visited[nx][ny] = True
                water += max(0, height - height_map[nx][ny])
                heapq.heappush(heap, (max(height, height_map[nx][ny]), nx, ny))
    return water
```

Complexity: $O(mn \log(mn))$ time, $O(m*n)$ space.

Pattern 10: Subsets / Backtracking

When to use it: You need to enumerate all possible combinations, permutations, or subsets, often with constraints. Backtracking builds candidates incrementally and abandons (“backtracks”) as soon as it determines a candidate can’t lead to a valid solution.

10.1 Subsets

```
def subsets(nums):
    result = [[]]
    for num in nums:
        result += [curr + [num] for curr in result]
    return result
```

Complexity: $O(n * 2^n)$ time, $O(n * 2^n)$ space.

10.2 Subsets II (With Duplicates)

```
def subsets_with_dup(nums):
    nums.sort()
    result = []

    def backtrack(start, path):
        result.append(list(path))
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return result
```

Complexity: $O(n * 2^n)$ time, $O(n * 2^n)$ space.

10.3 Permutations

```
def permute(nums):
    result = []

    def backtrack(path, remaining):
        if not remaining:
            result.append(list(path))
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i + 1:])
            path.pop()

    backtrack([], nums)
    return result
```

Complexity: $O(n * n!)$ time, $O(n * n!)$ space.

10.4 Permutations II (With Duplicates)

```
def permute_unique(nums):
    nums.sort()
    result = []
    used = [False] * len(nums)

    def backtrack(path):
        if len(path) == len(nums):
            result.append(list(path))
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue
            used[i] = True
            path.append(nums[i])
            backtrack(path)
            path.pop()
            used[i] = False

    backtrack([])
    return result
```

Complexity: $O(n * n!)$ time, $O(n * n!)$ space.

10.5 Combination Sum

Problem: Find all unique combinations from a set of numbers (reusable) that sum to a target.

```
def combination_sum(candidates, target):
    result = []
    candidates.sort()

    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(list(path))
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()

    backtrack(0, [], target)
    return result
```

Complexity: Exponential in general, bounded by the number of valid combinations.

10.6 Combination Sum II (No Reuse, With Duplicates)

```
def combination_sum2(candidates, target):
    candidates.sort()
    result = []

    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(list(path))
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            if candidates[i] > remaining:
                break
            path.append(candidates[i])
            backtrack(i + 1, path, remaining - candidates[i])
            path.pop()

    backtrack(0, [], target)
    return result
```

Complexity: Exponential in general, pruned by sorting and skipping duplicates.

10.7 Letter Case Permutation

Problem: Given a string with letters and digits, return all possible strings by switching letter cases.

```
def letter_case_permutation(s):
    result = [[]]
    for ch in s:
        if ch.isalpha():
            result = [path + [c] for path in result for c in (ch.lower(), ch.upper())]
        else:
            for path in result:
                path.append(ch)
    return "".join(path) for path in result
```

Complexity: $O(n * 2^m)$ time where m is the number of letters.

10.8 Generate Parentheses

```
def generate_parenthesis(n):
    result = []

    def backtrack(path, open_count, close_count):
        if len(path) == 2 * n:
            result.append("".join(path))
            return
        if open_count < n:
            path.append("(")
            backtrack(path, open_count + 1, close_count)
            path.pop()
        if close_count < open_count:
            path.append(")")
            backtrack(path, open_count, close_count + 1)
            path.pop()

    backtrack([], 0, 0)
    return result
```

Complexity: $O(4^n / \sqrt{n})$ time (Catalan number bound), same order space.

10.9 Word Search

Problem: Given a 2D board of letters and a word, determine if the word can be constructed from adjacent cells (no reuse).

```
def exist(board, word):
    rows, cols = len(board), len(board[0])

    def backtrack(r, c, index):
        if index == len(word):
            return True
        if r < 0 or r >= rows or c < 0 or c >= cols or board[r][c] != word[index]:
            return False
        temp, board[r][c] = board[r][c], "#"
        found = (backtrack(r + 1, c, index + 1) or backtrack(r - 1, c, index + 1) or
                 backtrack(r, c + 1, index + 1) or backtrack(r, c - 1, index + 1))
        board[r][c] = temp
        return found

    for r in range(rows):
        for c in range(cols):
            if backtrack(r, c, 0):
                return True
    return False
```

Complexity: $O(m * n * 4^L)$ time where L is word length, $O(L)$ space.

10.10 N-Queens

Problem: Place N queens on an N x N board such that no two queens attack each other.

```
def solve_n_queens(n):
    result = []
    cols, diag1, diag2 = set(), set(), set()
    board = []

    def backtrack(row):
        if row == n:
            result.append(list(board))
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            board.append(col)
            backtrack(row + 1)
            board.pop()
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0)
    return result
```

Complexity: $O(n!)$ worst case time, $O(n^2)$ space for output.

Pattern 11: Modified Binary Search

When to use it: Whenever the input is sorted (or has a monotonic property), and you're tempted to scan linearly — binary search almost always gets you to $O(\log n)$. This pattern covers variations beyond textbook binary search: unknown direction, ceilings/floors, rotated arrays, bitonic arrays, and searching answer-space.

11.1 Order-agnostic Binary Search

```
def binary_search(arr, key):
    start, end = 0, len(arr) - 1
    ascending = arr[start] <= arr[end]
    while start <= end:
        mid = (start + end) // 2
        if arr[mid] == key:
            return mid
        if ascending:
            if key < arr[mid]:
                end = mid - 1
            else:
                start = mid + 1
        else:
            if key > arr[mid]:
                end = mid - 1
            else:
                start = mid + 1
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space.

11.2 Ceiling of a Number

```
def search_ceiling_of_a_number(arr, key):
    if key > arr[-1]:
        return -1
    start, end = 0, len(arr) - 1
    while start <= end:
        mid = (start + end) // 2
        if key < arr[mid]:
            end = mid - 1
        elif key > arr[mid]:
            start = mid + 1
        else:
            return mid
    return start
```

Complexity: $O(\log n)$ time, $O(1)$ space.

11.3 Next Letter

Problem: Given a sorted circular list of letters, find the smallest letter strictly greater than a given key.

```
def next_letter(letters, key):
    start, end = 0, len(letters) - 1
    while start <= end:
        mid = (start + end) // 2
        if key < letters[mid]:
            end = mid - 1
        else:
            start = mid + 1
    return letters[start % len(letters)]
```

Complexity: $O(\log n)$ time, $O(1)$ space.

11.4 Number Range (First and Last Position)

```
def find_range(arr, key):
    def find_bound(is_first):
        start, end = 0, len(arr) - 1
        result = -1
        while start <= end:
            mid = (start + end) // 2
            if key < arr[mid]:
                end = mid - 1
            elif key > arr[mid]:
                start = mid + 1
            else:
                result = mid
                if is_first:
                    end = mid - 1
                else:
                    start = mid + 1
        return result
    return [find_bound(True), find_bound(False)]
```

Complexity: $O(\log n)$ time, $O(1)$ space.

11.5 Search in a Sorted Infinite Array

Problem: Given an array-like structure of unknown size, find a key.

Approach: Exponentially grow a search boundary until the key range is bracketed, then binary search within it.

```
def search_in_infinite_array(reader, key):
    start, end = 0, 1
    while reader.get(end) is not None and reader.get(end) < key:
        start = end
        end *= 2
    return binary_search_bounded(reader, key, start, end)

def binary_search_bounded(reader, key, start, end):
    while start <= end:
        mid = (start + end) // 2
        val = reader.get(mid)
        if val is None or val > key:
            end = mid - 1
        elif val < key:
            start = mid + 1
        else:
            return mid
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space (n = position of the key, or array length).

11.6 Minimum Difference Element

Problem: Find the element in a sorted array closest to a given key.

```
def search_min_diff_element(arr, key):
    if key <= arr[0]:
        return arr[0]
    if key >= arr[-1]:
        return arr[-1]
    start, end = 0, len(arr) - 1
    while start <= end:
        mid = (start + end) // 2
        if arr[mid] == key:
            return arr[mid]
        if arr[mid] < key:
            start = mid + 1
        else:
            end = mid - 1
    return arr[start] if arr[start] - key < key - arr[end] else arr[end]
```

Complexity: $O(\log n)$ time, $O(1)$ space.

11.7 Search Bitonic Array

Problem: Search a key in an array that increases then decreases.

```
def search_bitonic_array(arr, key):
    def find_max_index():
        start, end = 0, len(arr) - 1
        while start < end:
            mid = (start + end) // 2
            if arr[mid] > arr[mid + 1]:
                end = mid
            else:
                start = mid + 1
        return start

    def binary_search(start, end, ascending):
        while start <= end:
            mid = (start + end) // 2
            if arr[mid] == key:
                return mid
            if ascending:
                if key < arr[mid]:
                    end = mid - 1
                else:
                    start = mid + 1
            else:
                if key > arr[mid]:
                    end = mid - 1
                else:
                    start = mid + 1
        return -1

    max_index = find_max_index()
    result = binary_search(0, max_index, True)
    if result != -1:
        return result
    return binary_search(max_index + 1, len(arr) - 1, False)
```

Complexity: $O(\log n)$ time, $O(1)$ space.

11.8 Search in Rotated Sorted Array

```
def search_rotated_array(arr, key):
    start, end = 0, len(arr) - 1
    while start <= end:
        mid = (start + end) // 2
        if arr[mid] == key:
            return mid
        if arr[start] <= arr[mid]:
            if arr[start] <= key < arr[mid]:
                end = mid - 1
            else:
                start = mid + 1
        else:
            if arr[mid] < key <= arr[end]:
                start = mid + 1
            else:
                end = mid - 1
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space.

11.9 Find K Closest Elements

Problem: Find the K closest elements to a given value in a sorted array.

```
def find_k_closest_elements(arr, k, x):
    left, right = 0, len(arr) - k
    while left < right:
        mid = (left + right) // 2
        if x - arr[mid] > arr[mid + k] - x:
            left = mid + 1
        else:
            right = mid
    return arr[left:left + k]
```

Complexity: $O(\log(n - k))$ time, $O(k)$ space for the result.

11.10 Median of Two Sorted Arrays

Problem: Find the median of two sorted arrays in logarithmic time.

Approach: Binary search on the smaller array to find a valid partition point where the left halves combined have exactly $(n+1)//2$ elements.

```
def find_median_sorted_arrays(nums1, nums2):
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1
    m, n = len(nums1), len(nums2)
    low, high = 0, m
    while low <= high:
        partition1 = (low + high) // 2
        partition2 = (m + n + 1) // 2 - partition1
        left1 = nums1[partition1 - 1] if partition1 > 0 else float('-inf')
        left2 = nums2[partition2 - 1] if partition2 > 0 else float('-inf')
        right1 = nums1[partition1] if partition1 < m else float('inf')
        right2 = nums2[partition2] if partition2 < n else float('inf')
        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 0:
                return (max(left1, left2) + min(right1, right2)) / 2.0
            return max(left1, left2)
        elif left1 > right2:
            high = partition1 - 1
        else:
            low = partition1 + 1
```

Complexity: $O(\log(\min(m, n)))$ time, $O(1)$ space.

Pattern 12: Top K Elements (Heap)

When to use it: Whenever you need the K largest/smallest/most-frequent elements from a collection, without fully sorting it. Maintain a heap of size K: $O(n \log k)$ instead of $O(n \log n)$.

12.1 Top K Numbers

```
import heapq

def find_k_largest_numbers(nums, k):
    min_heap = nums[:k]
    heapq.heapify(min_heap)
    for num in nums[k:]:
        if num > min_heap[0]:
            heapq.heapreplace(min_heap, num)
    return min_heap
```

Complexity: $O(n \log k)$ time, $O(k)$ space.

12.2 Kth Smallest Number

```
import heapq

def find_kth_smallest_number(nums, k):
    max_heap = [-n for n in nums[:k]]
    heapq.heapify(max_heap)
    for num in nums[k:]:
        if -num > max_heap[0]:
            heapq.heapreplace(max_heap, -num)
    return -max_heap[0]
```

Complexity: $O(n \log k)$ time, $O(k)$ space.

12.3 K Closest Points to Origin

```
import heapq

def k_closest(points, k):
    max_heap = []
    for x, y in points:
        dist = -(x * x + y * y)
        if len(max_heap) < k:
            heapq.heappush(max_heap, (dist, x, y))
        elif dist > max_heap[0][0]:
            heapq.heapreplace(max_heap, (dist, x, y))
    return [[x, y] for _, x, y in max_heap]
```

Complexity: $O(n \log k)$ time, $O(k)$ space.

12.4 Connect Ropes with Minimum Cost

Problem: Connect N ropes of different lengths into one rope with minimum total cost, where the cost to connect two ropes equals their combined length.

```
import heapq

def minimum_cost_to_connect_ropes(ropes_lengths):
    heapq.heapify(ropes_lengths)
    total_cost = 0
    while len(ropes_lengths) > 1:
        a = heapq.heappop(ropes_lengths)
        b = heapq.heappop(ropes_lengths)
        cost = a + b
        total_cost += cost
        heapq.heappush(ropes_lengths, cost)
    return total_cost
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

12.5 Top K Frequent Numbers

```
import heapq
from collections import Counter

def top_k_frequent(nums, k):
    freq = Counter(nums)
    min_heap = []
    for num, count in freq.items():
        heapq.heappush(min_heap, (count, num))
        if len(min_heap) > k:
            heapq.heappop(min_heap)
    return [num for count, num in min_heap]
```

Complexity: $O(n \log k)$ time, $O(n)$ space.

12.6 Frequency Sort

Problem: Sort characters of a string by decreasing frequency.

```
import heapq
from collections import Counter

def frequency_sort(s):
    freq = Counter(s)
    max_heap = [(-count, ch) for ch, count in freq.items()]
    heapq.heapify(max_heap)
    result = []
    while max_heap:
        count, ch = heapq.heappop(max_heap)
        result.append(ch * (-count))
    return "".join(result)
```

Complexity: $O(n + d \log d)$ time where d is distinct characters, $O(n)$ space.

12.7 Kth Largest Number in a Stream

```
import heapq

class KthLargestStream:
    def __init__(self, k):
        self.k = k
        self.heap = []

    def add(self, num):
        heapq.heappush(self.heap, num)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

Complexity: $O(\log k)$ per insertion, $O(k)$ space.

12.8 K Closest Numbers (to a Given Value)

Problem: Given a sorted array, find K closest numbers to a given value X (heap-based alternative to 11.9).

```
import heapq

def find_closest_elements(arr, k, x):
    max_heap = []
    for num in arr:
        heapq.heappush(max_heap, (-abs(num - x), num))
        if len(max_heap) > k:
            heapq.heappop(max_heap)
    return sorted(num for _, num in max_heap)
```

Complexity: $O(n \log k)$ time, $O(k)$ space.

12.9 Maximum Distinct Elements

Problem: Given an array and K removals allowed, remove K elements to maximize the count of distinct numbers remaining.

```
from collections import Counter
import heapq

def maximum_distinct_elements(nums, k):
    freq = Counter(nums)
    distinct_count = 0
    min_heap = []
    for num, count in freq.items():
        if count == 1:
            distinct_count += 1
        else:
            heapq.heappush(min_heap, (count, num))
    while k > 0 and min_heap:
        count, num = heapq.heappop(min_heap)
        k -= count - 1
        if k >= 0:
            distinct_count += 1
    if k > 0:
        distinct_count -= k
    return distinct_count
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

12.10 Sum of Elements Between K1 and K2 Smallest

```
import heapq

def sum_between_k1_and_k2_smallest(nums, k1, k2):
    min_heap = list(nums)
    heapq.heapify(min_heap)
    for _ in range(k1):
        heapq.heappop(min_heap)
    total = 0
    for _ in range(k2 - k1 + 1):
        total += heapq.heappop(min_heap)
    return total
```

Complexity: $O(n \log n)$ time, $O(n)$ space.

Pattern 13: K-way Merge

When to use it: You have K sorted arrays/lists and need to merge them, or find a specific element (e.g., Kth smallest) across all of them, efficiently. Use a min-heap seeded with the first element of each list; repeatedly pop the smallest and push the next element from that same list.

13.1 Merge K Sorted Lists

```
import heapq

class ListNode:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

def merge_k_sorted_lists(lists):
    min_heap = []
    for i, head in enumerate(lists):
        if head:
            heapq.heappush(min_heap, (head.value, i, head))

    dummy = ListNode(0)
    curr = dummy
    while min_heap:
        val, i, node = heapq.heappop(min_heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(min_heap, (node.next.value, i, node.next))
    return dummy.next
```

Complexity: $O(N \log k)$ time (N = total nodes, k = number of lists), $O(k)$ space.

13.2 Kth Smallest Number in M Sorted Lists

```
import heapq

def find_kth_smallest(lists, k):
    min_heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(min_heap, (lst[0], i, 0))

    count, result = 0, None
    while min_heap:
        result, list_idx, elem_idx = heapq.heappop(min_heap)
        count += 1
        if count == k:
            return result
        if elem_idx + 1 < len(lists[list_idx]):
            heapq.heappush(min_heap, (lists[list_idx][elem_idx + 1], list_idx, elem_idx + 1))
    return result
```

Complexity: $O(k \log m)$ time (m = number of lists), $O(m)$ space.

13.3 Kth Smallest Number in a Sorted Matrix

```
import heapq

def kth_smallest_in_matrix(matrix, k):
    n = len(matrix)
    min_heap = []
    for r in range(min(k, n)):
        heapq.heappush(min_heap, (matrix[r][0], r, 0))
    number, count = -1, 0
    while min_heap:
        number, r, c = heapq.heappop(min_heap)
        count += 1
        if count == k:
            break
        if c + 1 < len(matrix[r]):
            heapq.heappush(min_heap, (matrix[r][c + 1], r, c + 1))
    return number
```

Complexity: $O(k \log(\min(k, n)))$ time, $O(\min(k, n))$ space.

13.4 Smallest Number Range Covering K Lists

Problem: Find the smallest range that includes at least one number from each of K sorted lists.

```
import heapq

def smallest_range(lists):
    min_heap = []
    current_max = float('-inf')
    for i, lst in enumerate(lists):
        heapq.heappush(min_heap, (lst[0], i, 0))
        current_max = max(current_max, lst[0])
    range_start, range_end = float('-inf'), float('inf')
    while len(min_heap) == len(lists):
        val, list_idx, elem_idx = heapq.heappop(min_heap)
        if current_max - val < range_end - range_start:
            range_start, range_end = val, current_max
        if elem_idx + 1 < len(lists[list_idx]):
            next_val = lists[list_idx][elem_idx + 1]
            heapq.heappush(min_heap, (next_val, list_idx, elem_idx + 1))
            current_max = max(current_max, next_val)
    return [range_start, range_end]
```

Complexity: $O(N \log k)$ time, $O(k)$ space.

13.5 K Pairs with Largest Sums

Problem: Given two sorted arrays, find the K pairs with the largest sums.

```
import heapq

def k_pairs_largest_sum(nums1, nums2, k):
    min_heap = []
    for i in range(min(k, len(nums1))):
        for j in range(min(k, len(nums2))):
            if len(min_heap) < k:
                heapq.heappush(min_heap, (nums1[i] + nums2[j], nums1[i], nums2[j]))
            elif nums1[i] + nums2[j] > min_heap[0][0]:
                heapq.heapreplace(min_heap, (nums1[i] + nums2[j], nums1[i], nums2[j]))
            else:
                break
    return [[a, b] for _, a, b in min_heap]
```

Complexity: $O(k^2 \log k)$ time (bounded search window), $O(k)$ space.

13.6 Merge K Sorted Arrays

```
import heapq

def merge_k_sorted_arrays(arrays):
    min_heap = []
    for i, arr in enumerate(arrays):
        if arr:
            heapq.heappush(min_heap, (arr[0], i, 0))
    result = []
    while min_heap:
        val, arr_idx, elem_idx = heapq.heappop(min_heap)
        result.append(val)
        if elem_idx + 1 < len(arrays[arr_idx]):
            heapq.heappush(min_heap, (arrays[arr_idx][elem_idx + 1], arr_idx, elem_idx + 1))
    return result
```

Complexity: $O(N \log k)$ time, $O(k)$ space.

13.7 Find K Pairs with Smallest Sums

```
import heapq

def k_smallest_pairs(nums1, nums2, k):
    if not nums1 or not nums2:
        return []
    min_heap = [(nums1[i] + nums2[0], i, 0) for i in range(min(k, len(nums1)))]
    heapq.heapify(min_heap)
    result = []
    while min_heap and len(result) < k:
        total, i, j = heapq.heappop(min_heap)
        result.append([nums1[i], nums2[j]])
        if j + 1 < len(nums2):
            heapq.heappush(min_heap, (nums1[i] + nums2[j + 1], i, j + 1))
    return result
```

Complexity: $O(k \log k)$ time, $O(k)$ space.

13.8 Kth Smallest Element in a BST

Problem: Find the Kth smallest element in a BST (in-order traversal is inherently a “merge” of already-sorted left/right subtrees).

```
def kth_smallest_in_bst(root, k):
    stack = []
    node = root
    count = 0
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        count += 1
        if count == k:
            return node.val
        node = node.right
    return -1
```

Complexity: $O(h + k)$ time, $O(h)$ space.

13.9 Median of K Sorted Arrays

Problem: Given K sorted arrays, find the median of all elements combined.

Approach: Merge using a heap (like 13.6) up to the midpoint(s) rather than fully materializing all elements when only the median is needed.

```

import heapq

def median_of_k_sorted_arrays(arrays):
    total = sum(len(a) for a in arrays)
    min_heap = []
    for i, arr in enumerate(arrays):
        if arr:
            heapq.heappush(min_heap, (arr[0], i, 0))
    merged = []
    while min_heap and len(merged) <= total // 2:
        val, arr_idx, elem_idx = heapq.heappop(min_heap)
        merged.append(val)
        if elem_idx + 1 < len(arrays[arr_idx]):
            heapq.heappush(min_heap, (arrays[arr_idx][elem_idx + 1], arr_idx, elem_idx + 1))
    if total % 2 == 1:
        return merged[total // 2]
    return (merged[total // 2 - 1] + merged[total // 2]) / 2.0

```

Complexity: $O(\text{total}/2 * \log k)$ time, $O(k)$ space (plus the partial merged output).

13.10 Find Common Elements in K Sorted Arrays

```

import heapq
from collections import defaultdict

def common_elements_k_arrays(arrays):
    min_heap = []
    for i, arr in enumerate(arrays):
        if arr:
            heapq.heappush(min_heap, (arr[0], i, 0))
    counts = defaultdict(int)
    result = []
    k = len(arrays)
    while min_heap:
        val, arr_idx, elem_idx = heapq.heappop(min_heap)
        counts[val] += 1
        if counts[val] == k:
            result.append(val)
        if elem_idx + 1 < len(arrays[arr_idx]):
            heapq.heappush(min_heap, (arrays[arr_idx][elem_idx + 1], arr_idx, elem_idx + 1))
    return result

```

Complexity: $O(N \log k)$ time, $O(N)$ space.

Pattern 14: Dynamic Programming (Knapsack Family)

When to use it: The problem asks for an optimal value (max/min/count) built from choices over items/elements, where choices have overlapping sub-problems (the same sub-problem recurs). Identify the state (what varies between sub-problems) and the transition (how a bigger sub-problem is built from smaller ones).

14.1 0/1 Knapsack

Problem: Given weights and values of items and a capacity, maximize value without exceeding capacity (each item used at most once).

```
def knapsack(profits, weights, capacity):
    n = len(profits)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for c in range(capacity + 1):
            dp[i][c] = dp[i - 1][c]
            if weights[i - 1] <= c:
                dp[i][c] = max(dp[i][c], profits[i - 1] + dp[i - 1][c - weights[i - 1]])
    return dp[n][capacity]
```

Complexity: $O(n * \text{capacity})$ time, $O(n * \text{capacity})$ space (reducible to $O(\text{capacity})$).

14.2 Equal Subset Sum Partition

Problem: Can the array be partitioned into two subsets with equal sum?

```
def can_partition(nums):
    total = sum(nums)
    if total % 2 != 0:
        return False
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for c in range(target, num - 1, -1):
            dp[c] = dp[c] or dp[c - num]
    return dp[target]
```

Complexity: $O(n * \text{target})$ time, $O(\text{target})$ space.

14.3 Subset Sum

Problem: Given a set of numbers, determine if there's a subset that sums to a given target.

```
def can_partition_target(nums, target):
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for c in range(target, num - 1, -1):
            dp[c] = dp[c] or dp[c - num]
    return dp[target]
```

Complexity: $O(n * \text{target})$ time, $O(\text{target})$ space.

14.4 Minimum Subset Sum Difference

Problem: Partition the array into two subsets so the difference of their sums is minimized.

```
def can_partition_min_diff(nums):
    total = sum(nums)
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for c in range(target, num - 1, -1):
            dp[c] = dp[c] or dp[c - num]
    closest = max(c for c in range(target, -1, -1) if dp[c])
    return total - 2 * closest
```

Complexity: $O(n * \text{total})$ time, $O(\text{total})$ space.

14.5 Count of Subset Sum

Problem: Count the number of subsets that sum to a given target.

```
def count_subsets(nums, target):
    dp = [0] * (target + 1)
    dp[0] = 1
    for num in nums:
        for c in range(target, num - 1, -1):
            dp[c] += dp[c - num]
    return dp[target]
```

Complexity: $O(n * \text{target})$ time, $O(\text{target})$ space.

14.6 Target Sum

Problem: Assign **+** or **-** to each number so the expression evaluates to a target; count the ways.

Approach: Reduces to Count of Subset Sum on a transformed target: $(\text{total} + \text{target}) / 2$.

```
def find_target_sum_ways(nums, target):
    total = sum(nums)
    if (total + target) % 2 != 0 or total < abs(target):
        return 0
    subset_target = (total + target) // 2
    dp = [0] * (subset_target + 1)
    dp[0] = 1
    for num in nums:
        for c in range(subset_target, num - 1, -1):
            dp[c] += dp[c - num]
    return dp[subset_target]
```

Complexity: $O(n * \text{total})$ time, $O(\text{total})$ space.

14.7 Unbounded Knapsack (Rod Cutting)

Problem: Like 0/1 Knapsack, but each item can be used unlimited times.

```
def unbounded_knapsack(profits, weights, capacity):
    n = len(profits)
    dp = [0] * (capacity + 1)
    for c in range(capacity + 1):
        for i in range(n):
            if weights[i] <= c:
                dp[c] = max(dp[c], profits[i] + dp[c - weights[i]])
    return dp[capacity]
```

Complexity: $O(n * \text{capacity})$ time, $O(\text{capacity})$ space.

14.8 Coin Change (Minimum Coins)

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for c in range(1, amount + 1):
        for coin in coins:
            if coin <= c:
                dp[c] = min(dp[c], dp[c - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

Complexity: $O(\text{amount} * \text{len}(\text{coins}))$ time, $O(\text{amount})$ space.

14.9 Coin Change II (Count Combinations)

```
def change(amount, coins):
    dp = [0] * (amount + 1)
    dp[0] = 1
    for coin in coins:
        for c in range(coin, amount + 1):
            dp[c] += dp[c - coin]
    return dp[amount]
```

Complexity: $O(\text{amount} * \text{len}(\text{coins}))$ time, $O(\text{amount})$ space.

14.10 Longest Common Subsequence

```
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

Complexity: $O(m * n)$ time, $O(m * n)$ space (reducible to $O(\min(m, n))$).

Pattern 15: Graphs (BFS / DFS / Union-Find / Topological Sort)

When to use it: Problems modeled as nodes and edges — grids, dependency chains, networks. The four core tools are: BFS (shortest paths / levels on unweighted graphs), DFS (connectivity, cycle structure), Union-Find (dynamic connectivity), and Topological Sort (ordering with dependencies).

15.1 Number of Islands

```
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != "1":
            return
        grid[r][c] = "0"
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                count += 1
                dfs(r, c)
    return count
```

Complexity: $O(m * n)$ time, $O(m * n)$ worst-case recursion space.

15.2 Clone Graph

```
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors or []

def clone_graph(node):
    if not node:
        return None
    visited = {}

    def dfs(n):
        if n in visited:
            return visited[n]
        copy = Node(n.val)
        visited[n] = copy
        for neighbor in n.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy

    return dfs(node)
```

Complexity: $O(V + E)$ time, $O(V)$ space.

15.3 Course Schedule (Cycle Detection via Topological Sort)

```

from collections import deque, defaultdict

def can_finish(num_courses, prerequisites):
    graph = defaultdict(list)
    in_degree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1
    queue = deque(c for c in range(num_courses) if in_degree[c] == 0)
    visited = 0
    while queue:
        node = queue.popleft()
        visited += 1
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)
    return visited == num_courses

```

Complexity: $O(V + E)$ time, $O(V + E)$ space.

15.4 Course Schedule II (Return the Order)

```

from collections import deque, defaultdict

def find_order(num_courses, prerequisites):
    graph = defaultdict(list)
    in_degree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1
    queue = deque(c for c in range(num_courses) if in_degree[c] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)
    return order if len(order) == num_courses else []

```

Complexity: $O(V + E)$ time, $O(V + E)$ space.

15.5 Number of Connected Components (Union-Find)

```

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        root_a, root_b = self.find(a), self.find(b)
        if root_a == root_b:
            return False
        if self.rank[root_a] < self.rank[root_b]:
            root_a, root_b = root_b, root_a
        self.parent[root_b] = root_a
        if self.rank[root_a] == self.rank[root_b]:
            self.rank[root_a] += 1
        return True

def count_components(n, edges):
    uf = UnionFind(n)
    components = n
    for a, b in edges:
        if uf.union(a, b):
            components -= 1
    return components

```

Complexity: $O((V + E) \log^* V)$ time (near $O(V + E)$ with union by rank + path compression), $O(V)$ space.

15.6 Graph Valid Tree

Problem: Given n nodes and a list of edges, determine if they form a valid tree.

Approach: A valid tree has exactly $n-1$ edges and is fully connected with no cycles — use Union-Find and reject any edge connecting two already-connected nodes.

```

def valid_tree(n, edges):
    if len(edges) != n - 1:
        return False
    uf = UnionFind(n)
    for a, b in edges:
        if not uf.union(a, b):
            return False
    return True

```

Complexity: $O(V + E)$ time, $O(V)$ space.

15.7 Redundant Connection

Problem: Given a tree with one extra edge added (creating a cycle), find the edge that can be removed.

```

def find_redundant_connection(edges):
    n = len(edges)
    uf = UnionFind(n + 1)
    for a, b in edges:
        if not uf.union(a, b):
            return [a, b]
    return []

```

Complexity: $O(n)$ time (with near-constant union/find), $O(n)$ space.

15.8 Word Ladder (BFS Shortest Transformation)

```

from collections import deque

def ladder_length(begin_word, end_word, word_list):
    word_set = set(word_list)
    if end_word not in word_set:
        return 0
    queue = deque([(begin_word, 1)])
    while queue:
        word, steps = queue.popleft()
        if word == end_word:
            return steps
        for i in range(len(word)):
            for c in "abcdefghijklmnopqrstuvwxyz":
                next_word = word[:i] + c + word[i + 1:]
                if next_word in word_set:
                    word_set.remove(next_word)
                    queue.append((next_word, steps + 1))
    return 0

```

Complexity: $O(N * L^2)$ time (N words, L word length), $O(N)$ space.

15.9 Rotting Oranges (Multi-source BFS)

```

from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c, 0))
            elif grid[r][c] == 1:
                fresh += 1
    minutes = 0
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    while queue:
        r, c, time = queue.popleft()
        minutes = max(minutes, time)
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, time + 1))
    return minutes if fresh == 0 else -1

```

Complexity: $O(m * n)$ time, $O(m * n)$ space.

15.10 Dijkstra's Shortest Path

Problem: Find the shortest path from a source to all other nodes in a weighted graph with non-negative edge weights.

```
import heapq
from collections import defaultdict

def dijkstra(n, edges, source):
    graph = defaultdict(list)
    for u, v, weight in edges:
        graph[u].append((v, weight))
        graph[v].append((u, weight))
    distances = [float('inf')] * n
    distances[source] = 0
    min_heap = [(0, source)]
    while min_heap:
        dist, node = heapq.heappop(min_heap)
        if dist > distances[node]:
            continue
        for neighbor, weight in graph[node]:
            new_dist = dist + weight
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                heapq.heappush(min_heap, (new_dist, neighbor))
    return distances
```

Complexity: $O((V + E) \log V)$ time, $O(V + E)$ space.

Closing Notes

You now have 150 worked problems across the 15 patterns that cover the overwhelming majority of coding-interview questions. A few tips for using this guide going forward:

- **Re-derive, don't memorize.** When you revisit a problem, try to rebuild the solution from the pattern's core idea rather than recalling the exact code.
- **Group by pattern when practicing.** Doing five "sliding window" problems back-to-back builds the pattern-recognition muscle much faster than jumping randomly between patterns.
- **Watch for hybrid problems.** Real interview questions often combine two patterns (e.g., Sliding Window + Two Pointers, or Backtracking + DP). Once each pattern is solid individually, practice spotting combinations.
- **Complexity first.** Before coding, state the target time/space complexity — it usually reveals which pattern is expected.

Good luck with your preparation.